

Estrutura de Dados — Revisão para a A1

Roteiro de revisão em etapas: Aulas 01 a 07

Disciplina: Estrutura de Dados **Curso:** Bacharelado em Sistemas de Informação — UNITINS

Professor: Alysson M. Bruno **Prova A1:** sexta-feira, 09/10/2026

Como usar este roteiro

Este texto é um **roteiro de revisão**. Ele está dividido em etapas numeradas, uma para cada assunto da prova. Faça uma etapa de cada vez, na ordem, com papel e caneta ao lado.

Cada etapa tem sempre as mesmas partes:

1. **Tempo estimado** — quanto a etapa costuma levar;
2. **Resumo direto** — o que você precisa saber, sem enrolação;
3. **Exemplo resolvido** — um exercício feito passo a passo;
4. **Armadilhas comuns** — os erros que mais tiram ponto;
5. **Perguntas de fixação** — tente responder **antes** de ler o “Confira a resposta”.

O roteiro inteiro leva **cerca de 3 horas**, contando o simulado. Você não precisa fazer tudo de uma vez: dividir em dois ou três dias funciona melhor do que uma noite só.

Etapa	Assunto	Tempo
0	Mapa da prova e autoavaliação	5 min
1	Compilar e executar Java: <code>javac</code> , <code>java</code> , <code>classpath</code> e pacotes	10 min
2	Classe abstrata, interface, <code>this</code> e polimorfismo	15 min
3	Análise de complexidade: contar as voltas dos laços	20 min
4	Recursão e pilha de chamadas	15 min
5	Arquivos em Java e TAD (contrato e JavaDoc)	15 min
6	Vetor dinâmico × lista encadeada e defeitos em código de lista	20 min
7	Pilha, fila e fila circular	15 min
8	Ordenação simples: Bubble, Selection e Insertion	20 min
9	Simulado, sem consulta, e correção	45 min + correção

Etapa 0 — Mapa da prova

Tempo estimado: 5 minutos.

Resumo direto

Item	Como é
Data	sexta-feira, 09/10/2026
Valor	10,0 pontos
Formato	individual, escrita, em papel e sem consulta
Parte I	10 questões objetivas , de 0,5 ponto cada (5,0 pontos), com alternativas A, B, C e D
Parte II	1 questão discursiva , de 5,0 pontos, respondida à mão, com caneta
Conteúdo	Aulas 01 a 07

O que cai (Aulas 01 a 07):

- compilação e execução em Java: javac, java, classpath e pacotes;
- classe abstrata, interface, this e polimorfismo;
- análise de complexidade: contar as voltas dos laços e estimar o tempo;
- recursão: caso base, passo recursivo, pilha de chamadas e traçado;
- arquivos em Java e TAD: contrato, pré-condição, pós-condição, invariante e JavaDoc;
- vetor dinâmico e lista encadeada: custos, escolha e efeito de um defeito no código;
- pilha, fila e fila circular com %;
- ordenação simples: Bubble Sort, Selection Sort e Insertion Sort.

O que NÃO cai: Merge Sort e Quick Sort (Aula 08), árvores e grafos.

Os formatos das questões objetivas. Você vai encontrar quatro jeitos de perguntar:

Formato	Como reconhecer	Como atacar
Traçado de código	“assinale o que é impresso” ou “descreva corretamente o resultado”	execute o código no papel, linha por linha, anotando as variáveis
Afirmções I a IV	“É correto apenas o que se afirma em”	marque V ou F em cada afirmação sozinha ; só depois procure a alternativa
Asserção-razão	duas frases ligadas por “PORQUE”	julgue I sozinha, julgue II sozinha e, só se as duas forem verdadeiras, pergunte: II explica I?
Escolha com justificativa	“assinale a decisão mais adequada e a justificativa correta”	a decisão e a justificativa têm de estar certas; muitas alternativas erradas têm a decisão certa e o motivo errado

Muitas questões pedem que você acompanhe um trecho de código à mão. Por isso, em todas as etapas deste roteiro, treine com lápis: escreva a tabela das variáveis e atualize a cada linha. Quem tenta

“adivinhar” o resultado olhando o código erra justamente nas armadilhas.

A **discursiva** pede para escrever à mão um método sobre lista, pilha ou fila e dar a complexidade dele em notação O , com justificativa. Ela é corrigida por cinco critérios: identificar o problema e os casos especiais (por exemplo, a estrutura vazia), montar a assinatura certa, desenvolver a lógica, interpretar a complexidade e escrever com clareza.

Autoavaliação: “eu sei...”

Leia cada frase e responda com sinceridade. Se a resposta for “não” ou “mais ou menos”, a última coluna diz para qual etapa voltar com mais calma.

Eu sei...	Se não, volte à
... escrever o comando <code>java</code> certo para uma classe que está em um pacote	Etapa 1
... dizer por que <code>new</code> de uma classe abstrata não compila, e o que <code>this.nome = nome</code> faz	Etapa 2
... contar as voltas de um laço com passo diferente de 1, ou de dois laços aninhados em que o de dentro vai até <code>i</code>	Etapa 3
... estimar o novo tempo quando <code>n</code> é multiplicado por 3 ou por 4	Etapa 3
... escrever, na ordem certa, o que um método recursivo imprime antes e depois da chamada	Etapa 4
... explicar o que é pré-condição, pós-condição e invariante, e quem garante cada uma	Etapa 5
... dizer por que o laço de leitura testa <code>!= null</code> , e não a linha vazia	Etapa 5
... dizer o custo de acessar o <code>k</code> -ésimo elemento em um <code>ArrayList</code> e em uma lista encadeada	Etapa 6
... desenhar a lista depois de <code>inserirNoInicio</code> , <code>inserirNoFim</code> e <code>remover</code>	Etapa 6
... achar o efeito de uma linha trocada ou esquecida em um método de lista	Etapa 6
... acompanhar <code>inicio</code> , <code>fim</code> e <code>tam</code> de uma fila circular, inclusive quando o índice volta para 0	Etapa 7
... escrever o vetor depois de uma passagem do Bubble, do Selection e do Insertion	Etapa 8
... dizer qual dos três é estável, qual faz menos trocas e qual é $O(n)$ no melhor caso	Etapa 8

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 1 — Compilar e executar Java: `javac`, `java`, `classpath` e pacotes

Tempo estimado: 10 minutos.

Resumo direto

- **javac** é o **compilador**: lê o arquivo `.java` e gera o arquivo `.class` (bytecode).
- **java** é o **lançador**: inicia a JVM e executa uma classe. Ele recebe o **nome da classe**, e não o nome do arquivo: nunca se escreve `.class` nem `.java` depois de `java`.
- **Pacote** é a “pasta lógica” da classe. A primeira linha `package br.unitins.revisao;` diz que a classe `0la` tem o **nome completo** `br.unitins.revisao.0la`.
- **Classpath** (opção `-cp`) diz à JVM **de qual pasta começar** a procurar as classes. A JVM junta o classpath com as pastas do pacote.
- **javac -d pasta** grava os `.class` dentro de pasta, **recriando as pastas do pacote**.

Exemplo resolvido

Projeto com o arquivo `fontes/br/unitins/revisao/0la.java`:

```
package br.unitins.revisao;

public class 0la {
    public static void main(String[] args) {
        System.out.println("Revisão da A1");
    }
}
```

Na raiz do projeto:

```
javac -d classes fontes/br/unitins/revisao/0la.java
java -cp classes br.unitins.revisao.0la
```

O que acontece, passo a passo:

1. `javac -d classes ...` cria o arquivo `classes/br/unitins/revisao/0la.class`. Repare: a pasta `classes` é a raiz; as pastas `br/unitins/revisao` vêm do package.
2. `java -cp classes br.unitins.revisao.0la` começa a procurar em `classes`, troca os pontos do nome completo por barras e acha `classes/br/unitins/revisao/0la.class`.
3. A saída é `Revisão da A1`.

O nome da pasta dos códigos-fonte (`fontes`) e o da pasta de saída (`classes`) são escolhas do projeto. O que não muda é a regra: **-cp aponta para a raiz; o nome da classe leva o pacote**.

Armadilhas comuns

Cada comando abaixo foi executado de verdade. Todos **falham**:

Comando errado	O que acontece	Por quê
<code>java -cp classes Ola</code>	Could not find or load main class Ola	falta o pacote no nome da classe
<code>java -cp classes br.unitins.revisao.Ola.class</code>	Could not find or load main class	java recebe nome de classe, sem .class
<code>java -cp classes/br/unitins/revisao Ola</code>	NoClassDefFoundError: Ola (wrong name: br/unitins/revisao/Ola)	o classpath deve ser a raiz , não a pasta do pacote
<code>javac -cp classes br.unitins.revisao.Ola</code>	error: Class names, 'br.unitins.revisao.Ola', are only accepted if annotation processing is explicitly requested	javac compila arquivo .java; quem executa é java

E um detalhe: sem a opção `-d`, o javac grava o `.class` **ao lado do .java**, dentro da pasta dos fontes.

Perguntas de fixação

1. O arquivo `fontes/br/unitins/cinema/Sessao.java` tem `package br.unitins.cinema;`. Depois de `javac -d build fontes/br/unitins/cinema/Sessao.java`, onde fica o `Sessao.class`?

Confira a resposta. Em `build/br/unitins/cinema/Sessao.class`. A pasta indicada em `-d` é a raiz, e as pastas do pacote são recriadas dentro dela.

2. Qual comando executa a classe da pergunta 1, a partir da raiz do projeto?

Confira a resposta. `java -cp build br.unitins.cinema.Sessao`. O classpath é `build` (a raiz) e o nome da classe é o nome completo, com o pacote e sem `.class`.

3. Uma colega digitou `java -cp build Sessao` e recebeu `Could not find or load main class`. O que ela esqueceu?

Confira a resposta. O pacote. Como a classe declara `package br.unitins.cinema;`, o nome dela é `br.unitins.cinema.Sessao`, e é esse nome que o java precisa receber.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 2 — Classe abstrata, interface, `this` e polimorfismo

Tempo estimado: 15 minutos.

Resumo direto

- **Interface** é um **contrato**: diz *o que* o objeto sabe fazer, sem dizer *como*. Quem escreve `implements` promete escrever todos os métodos do contrato.
- **Classe abstrata** é uma classe **incompleta**: pode ter atributos, métodos com corpo e métodos sem corpo. **Não pode ser instanciada**: `new ClasseAbstrata()` não compila. Mas pode ser o **tipo de**

uma variável ou de um vetor.

- Se a classe abstrata declara `implements Contrato` e não escreve um método do contrato, a **primeira subclasse concreta é obrigada** a escrevê-lo.
- Uma classe **estende uma só** classe, mas pode **implementar várias** interfaces.
- **this** é o próprio objeto. `this.nome = nome;` guarda o parâmetro `nome` no atributo `nome`. Sem o `this`, `nome = nome;` **compila**, mas não faz nada útil: o atributo fica `null`.
- **Polimorfismo**: o **tipo declarado** da variável decide **quais métodos podem ser chamados**; o **tipo real** do objeto (o que veio do `new`) decide **qual versão executa**.
- **Passagem de parâmetros**: Java passa sempre **uma cópia**. Para objetos, é uma cópia da **referência**: o método consegue mudar o estado do objeto, mas não consegue trocar a variável de quem chamou.

Exemplo resolvido

```
public interface Sonoro {
    String som();
}

public abstract class Instrumento implements Sonoro {
    protected String nome;

    public String getNome() { return nome; }
}

public class Violao extends Instrumento {
    public Violao(String nome) { this.nome = nome; }

    @Override
    public String som() { return "dlim"; }
}

public class Tambor extends Instrumento {
    public Tambor(String nome) { nome = nome; }    // falta o this

    @Override
    public String som() { return "tum"; }
}
```

```
Instrumento[] palco = { new Violao("Lia"), new Tambor("Davi") };
for (Instrumento x : palco) {
    System.out.println(x.getNome() + ": " + x.som());
}
```

Saída real:

```
Lia: dlim
null: tum
```

Por quê?

1. `Instrumento[]` `palco` compila: a classe abstrata pode ser **tipo** de vetor. O que não pode é `new Instrumento()`.
2. Em `x.som()`, o tipo declarado é `Instrumento`, mas cada objeto executa a **sua** versão: o violão faz “`dlim`”; o tambor, “`tum`”. Isso é polimorfismo.
3. No construtor de `Tambor`, `nome = nome;` atribui o parâmetro a ele mesmo. O atributo continua `null`, e o compilador não reclama.

Armadilhas comuns

Estas três linhas foram testadas no compilador e **não compilam**:

Código	Mensagem do compilador	Por quê
<code>Instrumento i = new Instrumento();</code>	<code>Instrumento is abstract; cannot be instantiated</code>	classe abstrata não gera objeto
<code>Sonoro s = new Violao("Téo"); s.getNome();</code>	<code>cannot find symbol</code>	o tipo declarado <code>Sonoro</code> só conhece <code>som()</code>
uma subclasse <code>Flauta</code> sem o método <code>som()</code>	<code>Flauta is not abstract and does not override abstract method som()</code>	a primeira classe concreta tem de cumprir o contrato

E duas que **compilam**, mas enganam:

- `nome = nome;` no construtor: compila e deixa o atributo `null`.
- `@Override` **não é obrigatório**. Ele só pede ao compilador que confira se o método realmente sobrescreve outro. Tirar o `@Override` de um método correto não quebra nada.

Perguntas de fixação

1. Com as classes do exemplo, o que imprime `Sonoro s = new Violao("Téo"); System.out.println(s.som());`?

Confira a resposta. Imprime `dlim`. A variável é do tipo `Sonoro`, mas o objeto é um `Violao`, e é a versão do objeto real que executa.

2. Verdadeiro ou falso: “como `Instrumento` já declara `implements Sonoro`, a classe `Violao` não precisa escrever `som()`”.

Confira a resposta. Falso. `Instrumento` declarou o contrato, mas não escreveu o método. Como `Violao` é concreta, ela é obrigada a escrever `som()`; sem ele, não compila.

3. Um método `static void renomear(Violao v) { v = new Violao("Gil"); }` é chamado com `renomear(meuViolao)`. O nome de `meuViolao` muda?

Confira a resposta. Não. O método recebeu uma **cópia da referência**. Fazer `v = new ...` só troca para onde a cópia aponta; a variável `meuViolao`, de quem chamou, continua apontando para o objeto

antigo. Se o método fizesse `v.nome = "Gil";`, aí sim o objeto mudaria, porque as duas referências apontam para o mesmo objeto.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 3 — Análise de complexidade: contar as voltas dos laços

Tempo estimado: 20 minutos.

Resumo direto

Para achar a complexidade de um trecho, siga **três passos**:

1. ache o laço que executa mais vezes (em geral, o mais de dentro);
2. **conte** quantas vezes o corpo dele roda, em função de n ;
3. **jogue fora** as constantes (o 2 de $2n$, o $1/2$ de $n/2$) e os termos menores.

Contagem	Complexidade	Se n for multiplicado por k , o tempo...
um número fixo (5, 7, 100)	$O(1)$	não muda
cerca de $\log_2 n$	$O(\log n)$	ganha só algumas voltas
n , $n/2$, $n/3$, $2n$, $3n$, $n + 10$	$O(n)$	é multiplicado por k
n^2 , $n^2/2$, $n(n - 1)/2$	$O(n^2)$	é multiplicado por k^2

A pergunta que resolve a maioria das questões: “se n dobrar (ou triplicar), o que acontece com o número de voltas?”

Exemplo resolvido: as cinco armadilhas

Todos os números abaixo foram conferidos por um programa que conta as voltas.

Armadilha 1 — passo diferente de 1.

```
for (int i = 0; i < n; i += 3) { ... }
```

O laço dá cerca de $n/3$ voltas (334 para $n = 1.000$; 1.334 para $n = 4.000$). Ainda é **$O(n)$** : dobrou n , dobrou o número de voltas. Andar “de 3 em 3” não muda a ordem de crescimento.

Armadilha 2 — laço de dentro que vai até i .

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) { ... }  
}
```

O de dentro roda 0, 1, 2, ..., $n - 1$ vezes. Somando: $n(n - 1)/2$ (499.500 para $n = 1.000$). É $O(n^2)$. Não se engane porque “o de dentro não vai até o fim”: metade de n^2 ainda é n^2 .

Armadilha 3 — laço de dentro que percorre só metade.

```
for (int i = 0; i < n; i++) {  
    for (int j = n / 2; j < n; j++) { ... }  
}
```

São $n \times n/2 = n^2/2$ voltas (500.000 para $n = 1.000$). Continua $O(n^2)$.

Armadilha 4 — limite fixo.

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < 3; j++) { ... }    // sempre 3 voltas  
}
```

São $3n$ voltas: $O(n)$, e não $O(n^2)$. Laço dentro de laço só dá n^2 quando **os dois** dependem de n . E um laço sozinho com limite fixo, como `for (int i = 0; i < 7; i++)`, dá sempre 7 voltas: $O(1)$, qualquer que seja n .

Armadilha 5 — laços em sequência (um depois do outro, não um dentro do outro).

```
for (int i = 0; i < n; i++) { ... }  
for (int j = 0; j < n; j++) { ... }
```

Laços em sequência **se somam**: $n + n = 2n$, que é $O(n)$. Se o segundo fosse um laço duplo, a soma seria $n + n^2$, e o termo que manda é o maior: $O(n^2)$.

Bônus — laço que dobra. `for (int i = 1; i < n; i *= 2)` dá cerca de $\log_2 n$ voltas (10 para $n = 1.000$; 20 para $n = 1.000.000$): $O(\log n)$.

Estimativa de tempo, passo a passo. Um trecho $O(n^2)$ levou 0,5 s com $n = 4.000$. Quanto levará com $n = 12.000$?

1. Fator de n : $12.000 \div 4.000 = 3$.
2. O trecho é $O(n^2)$, então o trabalho é multiplicado por $3^2 = 9$.
3. Tempo: $0,5 \text{ s} \times 9 = 4,5 \text{ s}$.

Se o mesmo trecho fosse $O(n)$, seria $0,5 \times 3 = 1,5 \text{ s}$; se fosse $O(1)$, continuaria 0,5 s.

Armadilhas comuns

- “Reduzi pela metade, então é $O(n/2)$.” Não existe $O(n/2)$: a constante $1/2$ vai embora e fica $O(n)$. Na estimativa de tempo, a metade aparece **nos dois tamanhos** e se cancela: ela não divide o fator.
- “Tem dois for, então é $O(n^2)$.” Só se um estiver **dentro** do outro e **os dois** dependerem de n .

- “ **$O(n^2)$ é só o pior caso, que quase nunca acontece.**” Nos laços que sempre rodam até o fim, o número de voltas não depende dos dados.
- Na estimativa, **eleve o fator ao quadrado** no caso $O(n^2)$: $n \times 3$ dá tempo $\times 9$, e não $\times 6$.

Perguntas de fixação

1. Qual a complexidade de `for (int i = 0; i < n; i += 5) soma += v[i];`?

Confira a resposta. $O(n)$. O laço dá cerca de $n/5$ voltas; tirando a constante $1/5$, fica n .

2. Um trecho com um laço duplo completo (i de 0 a n e j de 0 a n) levou 2 s com $n = 1.200$. Estime o tempo com $n = 3.600$.

Confira a resposta. O trecho é $O(n^2)$. O fator de n é 3; o trabalho é multiplicado por 9. Tempo: $2 \times 9 = 18$ s.

3. Qual a complexidade de um laço de 0 a n seguido de um laço de 0 a 10?

Confira a resposta. $O(n)$. A contagem é $n + 10$, e o termo que manda é n . O laço de limite fixo só acrescenta uma constante.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 4 — Recursão e pilha de chamadas

Tempo estimado: 15 minutos.

Resumo direto

- Um método **recursivo** chama a si mesmo com um problema **menor**.
- Todo método recursivo precisa de:
 - **caso base** — a situação que responde direto, **sem** nova chamada;
 - **passo recursivo** — a chamada com um problema menor, que **se aproxima** do caso base.
- Sem caso base (ou se o passo não se aproxima dele), as chamadas não param e a JVM interrompe o programa com **StackOverflowError**.
- Cada chamada ganha um **quadro** na **pilha de chamadas**. As chamadas **entram** (ida) até o caso base e depois **saem na ordem inversa** (volta).
- **Regra de ouro do traçado:** o que está **antes** da chamada recursiva roda **na ida**; o que está **depois** roda **na volta**, de trás para a frente.

Exemplo resolvido

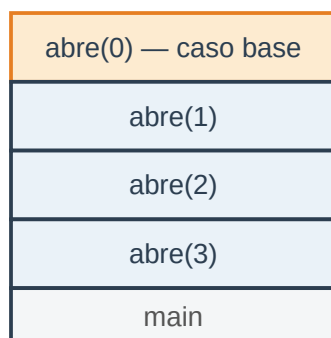
```
static void abre(int n) {  
    if (n == 0) {                // caso base  
        System.out.print("* ");  
        return;  
    }  
    System.out.print("(" + n + " "); // antes da chamada: na ida  
    abre(n - 1);                 // passo recursivo  
    System.out.print(n + " ");    // depois da chamada: na volta  
}
```

Traçado de `abre(3)`, com a pilha desenhada:

Momento	Pilha (topo à direita)	Imprime
entra <code>abre(3)</code>	<code>abre(3)</code>	(3
entra <code>abre(2)</code>	<code>abre(3)</code> <code>abre(2)</code>	(2
entra <code>abre(1)</code>	<code>abre(3)</code> <code>abre(2)</code> <code>abre(1)</code>	(1
entra <code>abre(0)</code> — caso base	<code>abre(3)</code> <code>abre(2)</code> <code>abre(1)</code> <code>abre(0)</code>	*
sai <code>abre(0)</code> , volta para <code>abre(1)</code>	<code>abre(3)</code> <code>abre(2)</code> <code>abre(1)</code>	1)
volta para <code>abre(2)</code>	<code>abre(3)</code> <code>abre(2)</code>	2)
volta para <code>abre(3)</code>	<code>abre(3)</code>	3)

Saída real: (3 (2 (1 * 1) 2) 3). Os parênteses mostram a pilha: cada chamada abre na ida e fecha na volta.

Pilha de chamadas de `abre(3)`



IDA (empilha, de baixo para cima):
imprime (3 (2 (1 e o caso base imprime *

VOLTA (desempilha, de cima para baixo):
imprime 1) 2) 3)

Saída: (3 (2 (1 * 1) 2) 3)

O que vem DEPOIS da chamada só roda na volta.

Pilha de chamadas de `abre(3)`: a ida empilha, a volta desempilha

Recursão que devolve valor. Para `static int dobra(int n) { if (n == 0) return 1; return 2 * dobra(n - 1); }`, a chamada `dobra(4)` vira `2 * dobra(3)`, que vira `2 * 2 * dobra(2)`, e assim até `dobra(0) = 1`. Na volta: 1, 2, 4, 8, **16**. O valor é montado **na volta**.

Armadilhas comuns

- **Imprimir na ordem da ida o que só roda na volta.** Se a impressão está depois da chamada, a ordem sai **invertida**: `soNaVolta(4)`, que só imprime depois de chamar `soNaVolta(n - 1)`, mostra 1 2 3 4, e não 4 3 2 1.
- **Esquecer que o caso base também pode imprimir.** No exemplo, o `*` aparece uma vez, no meio da saída.
- **Passo que pula o caso base.** Se o passo é `n - 3` e o caso base é `n == 0`, uma chamada com `n = 7` passa por 4, 1, `-2...` e nunca chega a 0. Por isso é comum escrever `n <= 0`.
- **Recursão não é mais rápida que laço.** O fatorial recursivo é $O(n)$ em tempo, como o iterativo, mas gasta $O(n)$ de memória na pilha.

Perguntas de fixação

1. O que imprime `abre(2)`?

Confira a resposta. `(2 (1 * 1) 2)`. Duas aberturas na ida, o caso base e dois fechamentos na volta.

2. Para `static void pulo(int n) { if (n <= 0) { System.out.print("fim "); return; } System.out.print(n + " "); pulo(n - 3); }`, o que imprime `pulo(7)`?

Confira a resposta. 7 4 1 fim. Tudo é impresso na ida: 7, 4, 1; a chamada seguinte recebe `-2`, que cai no caso base `n <= 0`.

3. Qual é o erro de `static int soma(int n) { return n + soma(n - 1); }`?

Confira a resposta. Falta o caso base. As chamadas nunca param (5, 4, 3, ..., `-1`, `-2...`), a pilha enche e a JVM lança `StackOverflowError` (conferido na execução). Corrigindo: `if (n == 0) return 0;` antes do `return`.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 5 — Arquivos em Java e TAD (contrato e JavaDoc)

Tempo estimado: 15 minutos.

Resumo direto: arquivos

- Para ler texto: `BufferedReader` envolvendo `FileReader`. O método `readLine()` devolve a próxima linha, ou **null no fim do arquivo**.
- Uma **linha em branco** chega como texto **vazio** (`""`), e **não** como `null`.
- **Cada chamada de `readLine()` consome uma linha.** Chame uma só vez por volta.
- **try-with-resources** (`try (recurso) { ... }`) fecha o arquivo sozinho ao sair do bloco, inclusive quando acontece uma exceção.

- `new FileWriter(caminho, StandardCharsets.UTF_8)` **apaga** o conteúdo anterior; com o terceiro argumento `true`, **acrescenta** no fim.
- Informe sempre `StandardCharsets.UTF_8`, para os acentos não virarem “JoÃ£o”.
- Processar **linha a linha** (ler, usar e descartar) gasta memória do tamanho de uma linha, e não do arquivo inteiro.
- `IOException` é **exceção verificada**: o compilador exige `try/catch` ou `throws`.

Resumo direto: TAD e JavaDoc

- **TAD** (Tipo Abstrato de Dados) diz **o que** a estrutura faz (as operações e o contrato), sem dizer **como**. Em Java, o TAD costuma ser uma **interface**; a classe que a implementa decide o como. Por isso dá para **trocar a implementação** (vetor por lista, por exemplo) sem mudar o código que usa o TAD.

	Quem garante	Quando vale
Pré-condição	quem chama a operação	antes da operação
Pós-condição	quem implementa a operação	depois da operação (se a pré foi cumprida)
Invariante	quem implementa	sempre, entre uma operação e outra

- Durante a execução de um método, o invariante pode ficar quebrado **por um instante**; ele tem de estar valendo de novo **antes** de o método terminar.
- **JavaDoc** é o comentário `/** ... */` que documenta o contrato: `@param`, `@return`, `@throws` e, por convenção da disciplina, `@pre`, `@pos` e `@invariant`.
- O compilador **não confere** o JavaDoc. Se a pré-condição precisa ser verificada, isso é feito **em código**, com `if` e `throw`.

Exemplo resolvido

Arquivo `inscritos.txt` com seis linhas: Lia, Davi, uma linha em branco, Nara, Otto e Iris.

```
int total = 0, vazias = 0;
try (BufferedReader leitor = new BufferedReader(
    new FileReader("inscritos.txt", StandardCharsets.UTF_8))) {
    String linha;
    while ((linha = leitor.readLine()) != null) {
        total++;
        if (linha.isBlank()) vazias++;
    }
}
System.out.println("linhas=" + total + " vazias=" + vazias);
```

Saída real: `linhas=6 vazias=1`. O laço só termina no `null` do fim do arquivo; a linha em branco é contada como linha comum. Se a condição fosse “parar na primeira linha vazia”, o programa leria só Lia e Davi (conferido).

Agora um TAD documentado, com o contrato escrito no Javadoc e conferido em código:

```
/**
 * TAD Estoque: quantidade de um produto guardado.
 *
 * <p><b>Invariante:</b> {@code saldo() >= 0}.</p>
 */
public interface Estoque {

    /**
     * Registra a saída de itens.
     *
     * @param qtd quantidade que saiu
     * @throws IllegalArgumentException se {@code qtd <= 0}
     * @throws IllegalStateException se {@code qtd > saldo()}
     * @pre {@code qtd > 0} e {@code qtd <= saldo()}
     * @pos {@code saldo()} diminuiu em {@code qtd}
     */
    void saida(int qtd);

    // ... entrada(int qtd) e saldo() seguem o mesmo padrão
}
```

Na implementação, a pré-condição vira código:

```
@Override
public void saida(int qtd) {
    if (qtd <= 0) throw new IllegalArgumentException("qtd deve ser positiva");
    if (qtd > saldo) throw new IllegalStateException("saldo insuficiente");
    saldo -= qtd;
}
```

Com saldo 6, a chamada `saida(9)` é recusada com `IllegalStateException` e o saldo continua 6 (conferido). O invariante `saldo() >= 0` foi protegido pela implementação.

Armadilhas comuns

- **`readLine()` duas vezes na mesma volta** (uma no `while`, outra no corpo): o programa pula metade das linhas.
- **Parar na linha vazia** em vez de parar no `null`: o programa termina antes do fim.
- **Achar que o Javadoc impede a chamada errada.** Ele documenta; quem impede é o código.
- **Trocar os papéis:** a pré-condição é dever de **quem chama**; a pós-condição e o invariante, de **quem implementa**.
- **Esquecer o `true` do `FileWriter`** quando a ideia é acrescentar: o conteúdo antigo some.

Perguntas de fixação

1. Em um laço de leitura, por que o teste é `!= null` e não `!linha.isEmpty()`?

Confira a resposta. Porque `readLine()` só devolve `null` no fim do arquivo. Uma linha em branco no meio devolve `" "`; parar nela faria o programa ignorar o resto do arquivo.

2. Um método `sacar(valor)` tem a pré-condição “ $\text{valor} \leq \text{saldo}$ ”. Quem deve garantir isso? E se o chamador não garantir?

Confira a resposta. Quem chama deve garantir. Se não garantir, a operação não promete nada; em Java, o usual é a implementação lançar uma exceção (`IllegalStateException` ou `IllegalArgumentException`), como no exemplo do Estoque.

3. Um programa grava "A" em `log.txt` com `new FileWriter("log.txt", StandardCharsets.UTF_8, false)` e depois grava "B" e "C" com o terceiro argumento `true`. O que fica no arquivo?

Confira a resposta. Três linhas: A, B e C (conferido). O `false` começou o arquivo do zero; cada `true` acrescentou no fim. Sem o `true`, cada gravação apagaria a anterior e sobraria só a última.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 6 — Vetor dinâmico × lista encadeada e defeitos em código de lista

Tempo estimado: 20 minutos.

Resumo direto: custos

Operação	Array fixo	Vetor dinâmico (ArrayList)	Lista simples (com cauda)	Lista dupla
Acesso pela posição (o k-ésimo)	O(1)	O(1)	O(n)	O(n)
Inserir no fim	O(1) se houver espaço	O(1) amortizado	O(1)	O(1)
Inserir no início	O(n)	O(n)	O(1)	O(1)
Remover o primeiro	O(n)	O(n)	O(1)	O(1)
Remover o último	O(1)	O(1)	O(n) — precisa achar o penúltimo	O(1)
Buscar um valor	O(n)	O(n)	O(n)	O(n)
Remover com a referência ao nó	—	—	O(n)	O(1)

- **Vetor dinâmico:** quando enche, cria um array com o **dobro** da capacidade e copia tudo ($O(n)$ naquela inserção). Como isso é raro, a inserção no fim custa **$O(1)$ amortizado**. Inserir ou remover **no meio** desloca os elementos seguintes: $O(n)$.
- **Lista encadeada:** cada **nó** guarda o dado e a referência proximo. A lista guarda cabeça (primeiro nó), cauda (último nó) e tamanho. Para chegar ao k-ésimo nó, é preciso **andar** desde a cabeça: $O(n)$.
- **Como escolher:** muito acesso pela posição → vetor (ArrayList, ou array se o tamanho nunca muda); inserir e remover nas **pontas** → lista; andar para trás ou remover um nó conhecido em $O(1)$ → lista dupla.

Exemplo resolvido: traçado em lista

Com a ListaEncadeada<T> da Aula 05 (métodos inserirNoInicio, inserirNoFim, remover, contem e imprimir, que mostra $e1 \rightarrow e2 \rightarrow \text{null}$ [tamanho=n]):

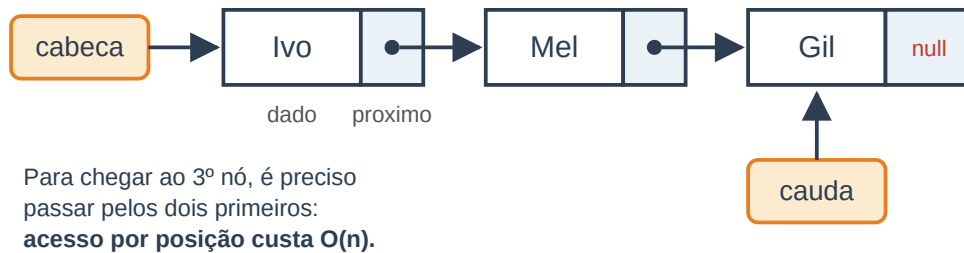
```
ListaEncadeada<String> fila = new ListaEncadeada<>();
fila.inserirNoFim("Mel");
fila.inserirNoFim("Gil");
fila.inserirNoInicio("Ivo");
fila.imprimir();
fila.remover("Gil");
fila.imprimir();
fila.inserirNoFim("Zeca");
fila.imprimir();
System.out.println(fila.contem("Gil"));
```

Saída real, com o desenho de cada passo:

```
Ivo → Mel → Gil → null  [tamanho=3]
Ivo → Mel → null  [tamanho=2]
Ivo → Mel → Zeca → null  [tamanho=3]
false
```

1. `inserirNoFim("Mel")` na lista vazia: cabeça e cauda passam a apontar para Mel.
2. `inserirNoFim("Gil")`: `cauda.proximo = Gil` e `cauda = Gil`.
3. `inserirNoInicio("Ivo")`: `Ivo.proximo = cabeça (Mel)` e depois `cabeça = Ivo`.
4. `remover("Gil")`: Gil era a cauda; o anterior (Mel) passa a ser a cauda e o `Mel.proximo` vira `null`.
5. `inserirNoFim("Zeca")` liga Zeca depois de Mel, que é a cauda certa.

Lista encadeada simples: Ivo → Mel → Gil → null [tamanho=3]



Lista encadeada depois das três primeiras inserções

Vetor dinâmico, conferido na execução. Com capacidade inicial 2, inserir cinco nomes faz a capacidade passar por $2 \rightarrow 4 \rightarrow 8$: ela dobra na 3ª e na 5ª inserção.

Exemplo resolvido: o efeito de um defeito

O `inserirNoInicio` correto primeiro liga o novo nó à cabeça antiga e **só depois** muda a cabeça. Veja o que acontece com as duas linhas trocadas:

```
public void inserirNoInicio(T dado) {  
    No<T> novo = new No<>(dado);  
    cabeca = novo;           // cabeça já mudou...  
    novo.proximo = cabeca;    // ...então o nó aponta para ele mesmo!  
    if (cauda == null) cauda = novo;  
    tamanho++;  
}
```

Depois de inserir "Mel" e "Gil", a cabeça é Gil e `Gil.proximo` é o próprio Gil. O nó de Mel ficou **perdido**, e o `imprimir()` entra em um laço sem fim: `Gil → Gil → Gil → ...` (conferido). O tamanho, porém, diz 2. Moral: **a ordem das ligações importa**; desenhe as setas antes de concluir.

Armadilhas comuns em código de lista

Quando a questão mostra um método “reescrito por um estudante”, **não confie no nome do método**: execute o código que está escrito, e não o que ele deveria fazer. Um roteiro que funciona:

1. desenhe os nós e as setas **antes** da primeira operação;
2. execute **uma linha por vez**, apagando e redesenhando cada seta que muda;
3. confira cabeça, cauda e tamanho depois de cada chamada;
4. teste mentalmente os casos de borda: lista **vazia**, lista com **um** nó, operação na **cabeça** e operação na **cauda**. É neles que os defeitos aparecem;
5. confira o que o laço de percurso examina: o primeiro nó? o último?

Defeitos típicos e o que eles provocam:

Defeito	Efeito
Esquecer <code>tamanho++</code> ou <code>tamanho--</code>	as setas estão certas, mas o <code>tamanho</code> impresso fica errado
Não tratar a lista vazia	<code>NullPointerException</code> ao usar <code>cabeca.dado</code> ou <code>ant.proximo</code>
Mudar a <code>cabeca</code> antes de ligar o novo nó	o resto da lista se perde
Percorrer usando a própria <code>cabeca</code> (<code>cabeca = cabeca.proximo</code>)	a lista é destruída; use uma variável auxiliar, como <code>atual</code>

Perguntas de fixação

1. Um sistema consulta a nota pela posição o tempo todo e quase nunca insere no meio. A quantidade de notas varia. Qual estrutura?

Confira a resposta. `ArrayList` (vetor dinâmico): acesso pela posição em $O(1)$ e tamanho variável. Na lista encadeada, cada consulta pela posição custaria $O(n)$.

2. Por que remover o **último** elemento de uma lista simples é $O(n)$, mesmo com cauda?

Confira a resposta. Porque, depois de remover, a cauda tem de passar para o **penúltimo** nó, e a lista simples não tem referência para trás: é preciso andar desde a cabeça até achar o nó cujo próximo é a cauda.

3. Suponha que o `inserirNoFim` tivesse esquecido o `tamanho++` (o `inserirNoInicio` continua certo). O que imprimiria o `imprimir()` depois das três primeiras inserções do exemplo resolvido (Mel e Gil no fim, Ivo no início)?

Confira a resposta. `Ivo → Mel → Gil → null` [`tamanho=1`] (conferido). As setas estão certas, porque o defeito não mexe em nenhuma ligação; só a inserção no início contou.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 7 — Pilha, fila e fila circular

Tempo estimado: 15 minutos.

Resumo direto

- **Pilha (LIFO — o último a entrar é o primeiro a sair):** `push(v)` põe no topo, `pop()` tira e devolve o topo, `peek()` só olha o topo. Na pilha com array da Aula 06, `topo` começa em `-1` (vazia); `push` faz `dados[++topo] = v`; `pop` faz `return dados[topo--]`; `tamanho()` é `topo + 1`. Usos: desfazer, verificar parênteses, pilha de chamadas.
- **Fila (FIFO — o primeiro a entrar é o primeiro a sair):** `enqueue(v)` põe no fim, `dequeue()` tira e devolve o da frente, `peek()` só olha a frente. Usos: atendimento, impressão.

- **Fila circular:** o array é tratado como um **anel**. Na Aula 06 a classe Fila usa `inicio` (quem sai), `fim` (onde o próximo entra) e `tam` (quantos há). Em outras versões o `inicio` se chama `frente` e o `tam` se chama `tamanho`; o funcionamento é o mesmo.
 - enqueue: `dados[fim] = v; fim = (fim + 1) % capacidade; tam++;`
 - dequeue: `v = dados[inicio]; inicio = (inicio + 1) % capacidade; tam--;`
 - o % faz o índice **voltar para 0** depois da última posição.
- Todas essas operações são **O(1)**: não percorrem nada.

Exemplo resolvido: pilha

Pilha com capacidade 5: `push(12)`, `push(7)`, `push(30)`, `pop()`, `push(4)`, `peek()`, `pop()`, `pop()` e `tamanho()`.

Operação	Pilha (topo à direita)	topo	Devolve
<code>push(12)</code>	12	0	
<code>push(7)</code>	12 7	1	
<code>push(30)</code>	12 7 30	2	
<code>pop()</code>	12 7	1	30
<code>push(4)</code>	12 7 4	2	
<code>peek()</code>	12 7 4	2	4 (não tira)
<code>pop()</code>	12 7	1	4
<code>pop()</code>	12	0	7
<code>tamanho()</code>	12	0	1

A saída do programa foi exatamente 30, 4, 4, 7 e 1.

Exemplo resolvido: fila circular

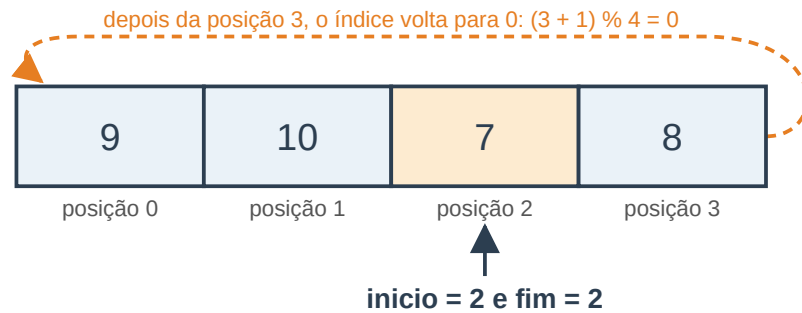
Fila com capacidade 4, começando vazia. Execute com lápis e compare.

Operação	inicio	fim	tam	array	Devolve
(início)	0	0	0	[_, _, _, _]	
<code>enqueue(5)</code>	0	1	1	[5, _, _, _]	
<code>enqueue(6)</code>	0	2	2	[5, 6, _, _]	
<code>enqueue(7)</code>	0	3	3	[5, 6, 7, _]	
<code>dequeue()</code>	1	3	2	[5, 6, 7, _]	5
<code>enqueue(8)</code>	1	0	3	[5, 6, 7, 8]	
<code>enqueue(9)</code>	1	1	4	[9, 6, 7, 8]	
<code>dequeue()</code>	2	1	3	[9, 6, 7, 8]	6
<code>enqueue(10)</code>	2	2	4	[9, 10, 7, 8]	

Pontos a observar (todos conferidos na execução):

1. Em enqueue(8), o fim era 3: $(3 + 1) \% 4 = 0$. O índice deu a volta.
2. O 9 foi gravado na **posição 0**, por cima do 5, que já tinha saído da fila.
3. Depois de um dequeue(), o valor **continua no array** (o 5 e o 6 ficaram lá), mas está **fora da fila**: o que vale são as posições a partir de início, contando tam elementos.
4. No fim, início e fim valem 2 e a fila está **cheia**. É por isso que a classe guarda tam: só com início == fim não dá para saber se a fila está cheia ou vazia.
5. A ordem de saída, esvaziando a fila, é **7, 8, 9, 10**: a ordem de chegada.

Fila circular com capacidade 4, depois de enqueue(10)



tam = 4: fila cheia. Ordem de saída: 7, 8, 9, 10 (sai sempre quem está em início).

Fila circular cheia, com início e fim na posição 2

Armadilhas comuns

- **Deixar o índice passar do fim.** Com capacidade 4, fim nunca vale 4: depois do 3 vem o 0.
- **Achar que dequeue apaga o valor do array.** Ele só avança início.
- **Confundir peek com pop/dequeue.** peek olha e não tira.
- **Esquecer os limites:** push em pilha cheia e enqueue em fila cheia lançam IllegalStateException; pop e dequeue na estrutura vazia também.
- **Trocar LIFO com FIFO:** na pilha sai o **último** que entrou; na fila, o **primeiro**.

Perguntas de fixação

1. Pilha com capacidade 2: push(1), push(2), push(3). O que acontece?

Confira a resposta. O terceiro push lança IllegalStateException("Pilha cheia."), porque topo já vale $1 = \text{dados.length} - 1$. Depois disso, dois pop() devolvem 2 e 1, e um terceiro pop() lança IllegalStateException("Pilha vazia.") (conferido).

2. Fila circular com capacidade 3: enqueue(1), enqueue(2), enqueue(3), dequeue() e enqueue(4). Em que posição do array fica o 4? Quanto valem início, fim e tam?

Confira a resposta. O 4 fica na **posição 0**: depois do terceiro enqueue, o fim já tinha voltado para 0. Ao final, início = 1, fim = 1, tam = 3 (fila cheia), e o próximo a sair é o 2 (conferido).

3. Qual estrutura verifica se os parênteses de $((a + b) * c)$ estão balanceados?

Confira a resposta. A pilha. Cada (é empilhado; cada) desempilha um. A expressão está balanceada se nunca faltar (para desempilhar e se a pilha terminar vazia.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 8 — Ordenação simples: Bubble, Selection e Insertion

Tempo estimado: 20 minutos.

Resumo direto

Algoritmo	O que faz em uma passagem	Melhor caso	Pior caso	Estável?	Trocas
Bubble Sort	compara vizinhos da esquerda para a direita e troca os que estão fora de ordem; o maior do trecho vai para o fim	$O(n)$ com a flag trocou	$O(n^2)$	sim	até $n(n - 1)/2$
Selection Sort	procura o menor do trecho não ordenado e faz uma única troca com a primeira posição do trecho	$O(n^2)$	$O(n^2)$	não	no máximo $n - 1$
Insertion Sort	pega $v[i]$ (a chave) e o insere no lugar certo do prefixo já ordenado, deslocando os maiores para a direita	$O(n)$ (vetor já ordenado)	$O(n^2)$	sim	deslocamentos, até $n(n - 1)/2$

- Os três são **in-place**: usam $O(1)$ de memória extra.
- Comparações do Selection Sort: sempre $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$, mesmo com o vetor já ordenado. Com $n = 5$, são 10; com $n = 6$, são 15.
- **Estável** = elementos iguais mantêm a ordem em que estavam. Bubble e Insertion são estáveis porque só mexem quando o valor é **estritamente maior** ($>$).
- **Como escolher**:
 - vetor quase ordenado ou dados chegando um a um → **Insertion Sort**;
 - memória que se desgasta com gravações (flash, EEPROM) → **Selection Sort** (poucas trocas);
 - empates precisam manter a ordem → um **estável** (Insertion ou Bubble), nunca o Selection;
 - os três são $O(n^2)$: para muitos dados, nenhum deles é boa escolha.

Exemplo resolvido: uma passagem de cada algoritmo

Vetor {6, 2, 9, 1, 5}. Os estados abaixo foram impressos pelo programa, passagem a passagem.

Bubble Sort, passagem i = 0 (j de 0 a 3, compara vizinhos):

j	compara	troca?	vetor depois
0	6 e 2	sim	[2, 6, 9, 1, 5]
1	6 e 9	não	[2, 6, 9, 1, 5]
2	9 e 1	sim	[2, 6, 1, 9, 5]
3	9 e 5	sim	[2, 6, 1, 5, 9]

Fim da passagem: [2, 6, 1, 5, 9], com 4 comparações e 3 trocas. O maior (9) chegou ao fim.

Selection Sort, passagem i = 0: procura o menor entre as posições 0 e 4: é o 1, na posição 3. Uma troca, entre v[0] e v[3]: [1, 2, 9, 6, 5]. Repare que o 6 foi para o lugar do 1 de uma vez, sem passar pelas posições do meio.

Insertion Sort, voltas i = 1 e i = 2:

- i = 1, chave = 2: o 6 é maior, desloca para a direita; o 2 entra na posição 0 → [2, 6, 9, 1, 5].
- i = 2, chave = 9: o 6 não é maior que 9; nada se desloca → [2, 6, 9, 1, 5].
- (Na volta i = 3, a chave 1 desloca 9, 6 e 2, e o vetor fica [1, 2, 6, 9, 5].)

Estabilidade na prática. Marque dois valores iguais: {5a, 5b, 2}. O Selection Sort acha o menor (2) e o troca com a posição 0: [2, 5b, 5a]. O 5b passou à frente do 5a: a troca entre posições afastadas **inverteu os iguais**. O Bubble e o Insertion nunca fariam isso.

Armadilhas comuns

- **No Bubble, a passagem para em $j < n - 1 - i$.** Na segunda passagem (i = 1) o último par não é comparado, porque o maior já está no fim.
- **O Bubble leva o maior para o fim**, e não o menor para o começo. O menor anda **uma posição por passagem** para a esquerda.
- **O Selection não troca com o vizinho.** Ele só **anota** o índice do menor e troca **uma vez** no fim da passagem, e só se $minIdx \neq i$.
- **O Insertion desloca, não troca.** Depois dos deslocamentos, a chave é escrita no buraco.
- **Bubble sem a flag trocou** faz todas as passagens mesmo com o vetor ordenado; só com a flag o melhor caso vira O(n).
- **“Os três são O(n²), então gastam o mesmo tempo.”** A notação descarta constantes: na prática, o Insertion Sort costuma ser bem mais rápido que o Bubble Sort.

Perguntas de fixação

1. Qual o vetor depois da **segunda** passagem (i = 1) do Bubble Sort em {6, 2, 9, 1, 5}?

Confira a resposta. Partindo de [2, 6, 1, 5, 9]: 2 e 6 não trocam; 6 e 1 trocam; 6 e 5 trocam; o par (6, 9) não é comparado. Resultado: [2, 1, 5, 6, 9] (conferido).

2. Quantas comparações e quantas trocas o Selection Sort faz no vetor {1, 2, 3, 4, 5}, já ordenado?

Confira a resposta. $4 + 3 + 2 + 1 = 10$ **comparações** e **nenhuma troca**: em toda passagem o menor já está na posição i , e o `if (minIdx != i)` impede a troca.

3. Uma lista de alunos já está em ordem alfabética e precisa ser ordenada por nota, mantendo a ordem alfabética entre notas iguais. Selection ou Insertion?

Confira a resposta. Insertion Sort, porque é estável: só desloca os **estritamente maiores**, e alunos com a mesma nota mantêm a ordem alfabética. O Selection Sort pode inverter empatados.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 9 — Simulado

Tempo estimado: 45 minutos para fazer, mais o tempo de corrigir.

Agora é a hora de testar tudo **como na prova**. Abra o arquivo **Simulado** desta aula, publicado no Classroom junto deste texto (10 objetivas e 1 discursiva).

1. **Sem consulta.** Feche este texto, os slides e o caderno. O simulado mostra o que você já sabe fazer sozinho; consultar esconde exatamente o que precisa ser revisto.
2. **Discursiva à mão, no papel, com caneta.** A prova é manuscrita. Escrever código à mão é diferente de digitar: ninguém avisa que faltou um ponto e vírgula, uma chave ou o `return`, e não há como apagar e reorganizar com facilidade. Treinar no papel agora é o que evita a surpresa no dia.
3. **Código no papel.** Nas objetivas com código, faça a tabela das variáveis. É assim que se acerta traçado.
4. **Só depois de terminar,** leia o capítulo seguinte, “Resolução comentada do simulado”, e corrija você mesmo. Para cada erro, volte à etapa indicada e refaça as perguntas de fixação.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Resolução comentada do simulado

Faça o simulado antes de ler esta parte. Ler a resolução antes de tentar transforma o simulado em leitura, e leitura não treina para a prova.

Respostas das objetivas

Questão	1	2	3	4	5	6	7	8	9	10
Resposta	D	B	A	C	A	D	B	C	D	A
Etapas	1	2	3	4	5	6	6	7	8	8

Conte seus acertos. Cada objetiva vale 0,5 na prova: 7 acertos equivaleriam a 3,5 dos 5,0 pontos da Parte I.

Questão 1 — resposta D

Com `-d` saída, o `javac` usa saída como raiz e recria dentro dela as pastas do pacote `br.unitins.academia`. O arquivo fica em **saída/br/unitins/academia/Treino.class**. É por isso que a execução usa `-cp` saída e o nome completo `br.unitins.academia.Treino`.

- **A** está errada: ignora o pacote; uma classe com package sempre fica dentro das pastas do pacote.
- **B** está errada: é onde o `.class` ficaria **sem** a opção `-d`.
- **C** está errada: inverte a ordem; a pasta de saída é a raiz, e o pacote fica dentro dela.

Questão 2 — resposta B

A soma é $120 + 90 + 120 = 330$: cada objeto executa a sua versão de `mensalidade()`.

- **I** é verdadeira.
- **II** é verdadeira: classe abstrata pode ser tipo de variável e de vetor; o proibido é `new Aluno()`.
- **III** é falsa: o tipo declarado decide **quais** métodos podem ser chamados; quem decide **qual versão** executa é o objeto real. Por isso Davi paga 90.
- **IV** é falsa: `nome = nome`; compila; só deixa o atributo `null`. A soma continuaria 330.

Corretas: I e II. **A** e **C** incluem a III; **D** inclui a IV.

Questão 3 — resposta A

O laço vai de $n - 1$ até $n / 2$: são $n/2$ voltas. Com $n = 2.500$, são 1.250; com $n = 7.500$, 3.750. O número de voltas foi multiplicado por 3, e o tempo também: $5\text{ s} \times 3 = 15\text{ s}$. O trecho é $O(n)$.

- **B** está errada: é a armadilha da metade. A metade aparece nos dois tamanhos e se cancela; ela não divide o fator.
- **C** está errada: não há laço dentro de laço.
- **D** está errada: andar para trás não muda nada; o número de voltas depende de n .

Questão 4 — resposta C

As chamadas entram assim: `somaDigitos(472) → somaDigitos(47) → somaDigitos(4)`. Só o caso base imprime na ida: 4. Na volta, a chamada de 47 calcula $7 + 4 = 11$ e imprime 11; a de 472 calcula $2 + 11 = 13$ e imprime 13. Saída: **4 11 13**.

- **A** mostra a ordem de entrada na pilha, mas as impressões acontecem na volta.
- **B** soma da direita para a esquerda, como se cada chamada imprimisse antes de chamar a próxima.
- **D** imprime os argumentos, que o método não imprime.

Questão 5 — resposta A

As duas asserções são verdadeiras, e a II explica a I. O `while (leitor.readLine() != null)` lê a 1ª linha só para testar e a joga fora; o corpo lê e imprime a 2ª. Na volta seguinte, o teste gasta a 3ª e o corpo imprime a 4ª, e assim por diante. Saem o 2º, o 4º e o 6º pedido.

- **B** está errada: o `try-with-resources` fecha o arquivo uma única vez, ao sair do bloco.
- **C** está errada: cada chamada de `readLine()` avança uma linha.
- **D** está errada: `readLine()` avança sempre, guardando ou não o resultado.

Questão 6 — resposta D

As operações são: inserir no fim, retirar o último e acessar pela posição. O `ArrayList` faz as três em $O(1)$: inserção no fim $O(1)$ amortizado, remover o último sem deslocar ninguém e acesso pela posição direto.

- **A** é a armadilha: inserir no fim é $O(1)$ com cauda, mas **retirar o último** na lista simples é $O(n)$, porque é preciso achar o penúltimo. E o acesso pela posição também é $O(n)$.
- **B** está errada: array não cresce sozinho, e a quantidade varia.
- **C** está errada: qualquer lista consegue retirar o último; além disso, a lista dupla continua com acesso pela posição em $O(n)$.

Questão 7 — resposta B

O método perdeu o caso especial da cabeça. O laço compara sempre `ant.proximo.dado`, e `ant` começa na cabeça: o dado da própria cabeça **nunca** é comparado.

- `remover("Lia")`: compara Davi e Nara, não acha Lia e devolve `false`.
- `remover("Nara")`: com `ant` em Davi, `ant.proximo` é Nara; como Nara era a cauda, cauda passa a ser Davi; devolve `true`.
- `imprimir()`: `Lia → Davi → null` [tamanho=2].
- **A** está errada: a cabeça não é removida.
- **C** está errada: o último nó é examinado, porque o teste olha `ant.proximo`.
- **D** está errada: a lista não está vazia; `ant` começa em Lia. (Na lista vazia, sim, este método lançaria `NullPointerException`: é o segundo defeito dele.)

Questão 8 — resposta C

Operação	inicio	fim	tam	array
enqueue de 11, 22, 33 e 44	0	4	4	[11, 22, 33, 44, _]
dequeue() → 11	1	4	3	
dequeue() → 22	2	4	2	
enqueue(55)	2	0	3	[11, 22, 33, 44, 55]
enqueue(66)	2	1	4	[66, 22, 33, 44, 55]
enqueue(77)	2	2	5	[66, 77, 33, 44, 55]
dequeue() → 33	3	2	4	

- **I** é verdadeira: o último dequeue() devolve 33.
- **II** é falsa: com capacidade 5, fim nunca passa de 4; $(4 + 1) \% 5 = 0$. Ao final vale 2.
- **III** é verdadeira: o 66 entrou quando fim já tinha voltado para 0, a posição em que o 11 estava (o 11 já tinha saído da fila).
- **IV** é falsa: a fila encheu depois do enqueue(77), mas o último dequeue() deixou tam = 4.

Corretas: I e III.

Questão 9 — resposta D

No vetor {1, 2, 3, 4, 5, 6}, já ordenado:

- **Selection Sort:** $5 + 4 + 3 + 2 + 1 = 15$ comparações e **nenhuma troca**. Ele não tem como parar antes: percorre todo o trecho em toda passagem.
- **Insertion Sort:** em cada volta, o while compara uma vez, vê que o anterior não é maior e para. São **5 comparações** e **nenhum deslocamento**: é o melhor caso, $O(n)$.

Trocar ajuda. **A** ignora que o Insertion Sort se adapta à entrada; **B** inventa deslocamentos que não acontecem; **C** atribui ao Selection Sort uma parada antecipada que ele não tem.

Questão 10 — resposta A

Passagem $i = 0$: [4, 7, 2, 6, 9] (o 9 vai para o fim). Passagem $i = 1$, com j de 0 a 2: 4 e 7 não trocam; 7 e 2 trocam; 7 e 6 trocam → [4, 2, 6, 7, 9]. A asserção I é verdadeira.

A asserção II é falsa: o Bubble Sort leva o **maior** valor do trecho para o **fim**. O 2 anda só uma posição para a esquerda por passagem (da posição 3 para a 2, depois para a 1).

- **B** está errada porque a II é falsa.
- **C** está errada: [2, 4, 6, 7, 9] só aparece ao fim da terceira passagem.
- **D** está errada: [4, 7, 2, 6, 9] é o estado depois da **primeira** passagem, e o Bubble compara vizinhos.

Questão 11 — resposta-modelo

Este código foi compilado e testado com a lista vazia, com uma lista de três nós (índices 0, 1 e 2), com os índices 3 e -1 e com uma lista de um só nó.

```
/**
 * Devolve o dado guardado na posição {@code indice} (0 é a cabeça).
 *
 * @param indice posição procurada, de 0 a tamanho - 1
 * @return o dado do nó naquela posição
 * @throws IndexOutOfBoundsException se o índice for inválido
 *         (inclusive quando a lista está vazia)
 */
public T obter(int indice) {
    if (indice < 0 || indice >= tamanho) {          // cobre a lista vazia
        throw new IndexOutOfBoundsException("Índice inválido: " + indice);
    }
    No<T> atual = cabeca;                          // começa no primeiro nó
    for (int i = 0; i < indice; i++) {              // anda 'indice' vezes
        atual = atual.proximo;
    }
    return atual.dado;
}
```

Por que funciona:

- **Lista vazia:** tamanho é 0, então qualquer índice cai em `indice >= tamanho` e a exceção é lançada antes de tocar em `cabeca`, que é `null`.
- **Percurso:** a variável `atual` começa na cabeça e avança exatamente `indice` vezes. Para `indice = 0`, o laço não roda e devolve o dado da cabeça.
- **Retorno:** devolve `atual.dado`, o dado, e não o nó.
- **Não altera a lista:** quem anda é `atual`; `cabeca`, `cauda` e `tamanho` não mudam.

b) Complexidade. O trecho mais executado é `atual = atual.proximo`, que roda `indice` vezes. No pior caso, o índice é o último, $n - 1$, e o laço roda $n - 1$ vezes: **$O(n)$** . No melhor caso, `indice = 0`: o laço não roda e o custo é **$O(1)$** . É o motivo de a lista perder para o vetor no acesso pela posição: no `VetorDinamico`, o `obter` é sempre $O(1)$.

Também vale uma versão com `while` e contador, ou uma que teste `cabeca == null` antes de testar o índice, desde que o índice negativo e o índice maior ou igual ao tamanho também sejam recusados.

Corrija a sua discursiva com este barema

Item	O que conferir na sua resposta	Pontos
1. Assinatura	<code>public T obter(int indice)</code> (o <code>public</code> é opcional), devolvendo <code>T</code>	0,5
2. Lista vazia e índice inválido	recusa índice negativo e índice maior ou igual a <code>tamanho</code> , lançando <code>IndexOutOfBoundsException</code> ; na lista vazia, não chega a usar <code>cabeca</code>	1,0
3. Percurso com o ponteiro	uma variável auxiliar começa em <code>cabeca</code> e avança <code>atual = atual.proximo</code> exatamente <code>indice</code> vezes	1,5
4. Retorno	devolve <code>atual.dado</code> e não mexe em <code>cabeca</code> , <code>cauda</code> nem <code>tamanho</code>	0,5
5. Complexidade justificada	$O(n)$ no pior caso, dizendo que o laço roda $n - 1$ vezes quando o índice é o último; melhor caso $O(1)$	1,0
6. Clareza	código organizado, nomes claros, justificativa objetiva	0,5
	Total	5,0

Erros que mais tiram ponto (todos testados):

- **Laço com `i <= indice`:** anda uma vez a mais. Devolve o dado do nó **seguinte** e, no último índice, lança `NullPointerException` (item 3).
- **Andar com a própria `cabeca`** (`cabeca = cabeca.proximo`): devolve o dado certo, mas destrói a lista (item 4).
- **`return atual;`** não compila, porque `atual` é um `No<T>`, e o método promete devolver `T` (item 4).
- **Sem testar o índice:** na lista vazia, `atual.dado` lança `NullPointerException` (item 2).
- **“É $O(1)$, porque é acesso por índice”:** isso vale para vetor, não para lista (item 5).
- **Só “ $O(n)$ ”, sem dizer quantas vezes o laço roda:** a justificativa vale metade do item 5.

E agora? Para cada questão errada, volte à etapa indicada na tabela das respostas e refaça as perguntas de fixação dela. Se sobrar tempo, refaça o simulado inteiro daqui a dois dias, sem olhar esta resolução.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Referências

ASCENCIO, Ana Fernanda Gomes; ARAÚJO, Graziela Santos de. **Estruturas de dados:** algoritmos, análise da complexidade e implementações em Java e C/C++. 1. ed. São Paulo: Pearson, 2010. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 24 mar. 2024.

BLOCH, Joshua. **Java efetivo**. 3. ed. Rio de Janeiro: Alta Books, 2019.

CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DEITEL, Paul; DEITEL, Harvey. **Java: como programar**. 10. ed. São Paulo: Pearson, 2016.

GUIMARÃES, Angelo de Moura. **Algoritmos e estruturas de dados**. Rio de Janeiro: LTC, 2008.

ORACLE. **Java Platform, Standard Edition & Java Development Kit**: API specification. Versão 25. Disponível em: <https://docs.oracle.com/en/java/javase/25/docs/api/>. Acesso em: 10 set. 2026.

PREISS, Bruno R. **Estrutura de dados e algoritmos**: padrões de projetos orientados a objetos com Java. São Paulo: Elsevier, 2000.

PUGA, Sandra Gavioli; RISSETTI, Gerson. **Lógica de programação e estrutura de dados**: com aplicações em Java. 2. ed. São Paulo: Pearson, 2009. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 24 mar. 2024.

SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. **Estruturas de dados e seus algoritmos**. Rio de Janeiro: LTC, 2010.